



ECE-560, Fall 2025

Assertion Based Verification

Asst. Prof of Practice: Venkatesh Patil

Maseeh College of Engineering and Computer Science

Final Project Report

Formal Verification of Asynchronous FIFO

Team Number #19

Aadityasingh Chouhan [aadityac@pdx.edu]

Manank Bhavsar [mbhavsar@pdx.edu]

- Table of Contents
- Introduction
- Block Diagram
- Timing Diagrams
- Formal Verification test plan
- Explanation of Assumptions used
- Explanation of Assertions used
- Explanation of Cover used
- Your analysis/Waveforms
- Bug Injection
- Execution of Different Failure Analysis
- Challenges Faces
- Conclusion
- Credits, Citing, References used
- Appendix (Any supplement information you like to add)

Introduction

Asynchronous FIFOs are fundamental building blocks in modern digital systems, enabling reliable data transfer between independent and unrelated clock domains. Because the write and read sides operate on different clocks, the design must handle metastability, pointer synchronization, and correct full/empty detection under all possible timing relationships. Any flaw in these areas can lead to data corruption, overflow/underflow conditions, or system deadlock. Ensuring the correctness of such structures is therefore essential, especially in SoC and high-performance FPGA/ASIC designs where clock domain crossings are common.

Traditional simulation approaches are inadequate for verifying asynchronous structures because they cannot exhaustively explore all possible phase shifts, frequency differences, and interleaving between the write and read operations. Even extensive randomized testing cannot guarantee coverage of all corner-case timing interactions. For this reason, assertion-based formal verification provides a mathematically complete and exhaustive method to validate correctness. Using System Verilog Assertions (SVA) together with Synopsys VC Formal, this project verifies safety, functional correctness, and CDC integrity of an asynchronous FIFO design.

The purpose of this project is to formally verify an RTL implementation of a parameterized asynchronous FIFO. The design includes Gray-coded write/read pointers, dual-flop synchronizers for pointer transfer, and flag generation logic for full and empty conditions. The verification approach involves building a set of assumptions to constrain legal operating conditions, writing formal assertions that encode the FIFO's required behaviour, and constructing cover properties to demonstrate the reachability of key operational states such as FIFO full, FIFO empty, pointer wrap-around, and concurrent read/write activity.

Through formal verification, the project demonstrates how assertion-based methodologies can expose subtle design flaws that simulation might miss, while also providing strong mathematical guarantees about the absence of incorrect behaviour. This introduction sets the stage for the subsequent sections, which describe the design architecture, formal properties, verification methodology, bug-injection analysis, and final conclusions drawn from the verification results.

Design description

The asynchronous FIFO design under verification is a dual-clock buffer used to safely transfer data between two independent clock domains: the write domain (wclk) and the read domain (rclk). Because these two clocks do not share a fixed frequency or phase relationship, the design incorporates specialized circuitry to prevent metastability, ensure correct pointer advancement, and generate reliable full and empty status signals.

The FIFO is structurally composed of five major RTL modules:

1. Write Pointer and Full Logic (wptr_full.sv)
 - Maintains the binary and Gray-coded write pointers (wbin, wptr).
 - Increments the pointer only when winc is asserted and the FIFO is not full.
 - Generates the wfull signal using a Gray-code comparison between the local write pointer and the synchronized read pointer (wq2_rptr).
 - Implements the standard inverted-MSB technique for full detection, ensuring correctness on wrap-around.

2. Read Pointer and Empty Logic (rptr_empty.sv)

- Maintains the binary and Gray-coded read pointers (rbin, rptr).
- Updates the pointer on valid read operations when the FIFO is not empty.
- Computes the rempty flag by comparing the local read pointer with the synchronized write pointer (rq2_wptr).
- Ensures correct empty detection even under asynchronous updates.

3. FIFO Memory Array (fifomem.sv)

- Implements the physical storage array accessed independently by the write and read processes.
- Write operations occur in the wclk domain using the binary write address.
- Read operations occur in the rclk domain using the binary read address.
- Guarantees data ordering (first-in, first-out) while supporting simultaneous read and write to different addresses.

4. Pointer Synchronizers (sync_r2w.sv and sync_w2r.sv)

- Each synchronizer consists of a classic two-stage flip-flop pipeline used to safely transfer Gray-coded pointers across domains.
- sync_r2w transfers the read pointer into the write clock domain, while sync_w2r transfers the write pointer into the read domain.
- The use of Gray code ensures that only one bit changes per update, reducing the risk of metastability and guaranteeing predictable synchronized values.

5. Top-Level FIFO Wrapper (asynch_fifo_dut.sv)

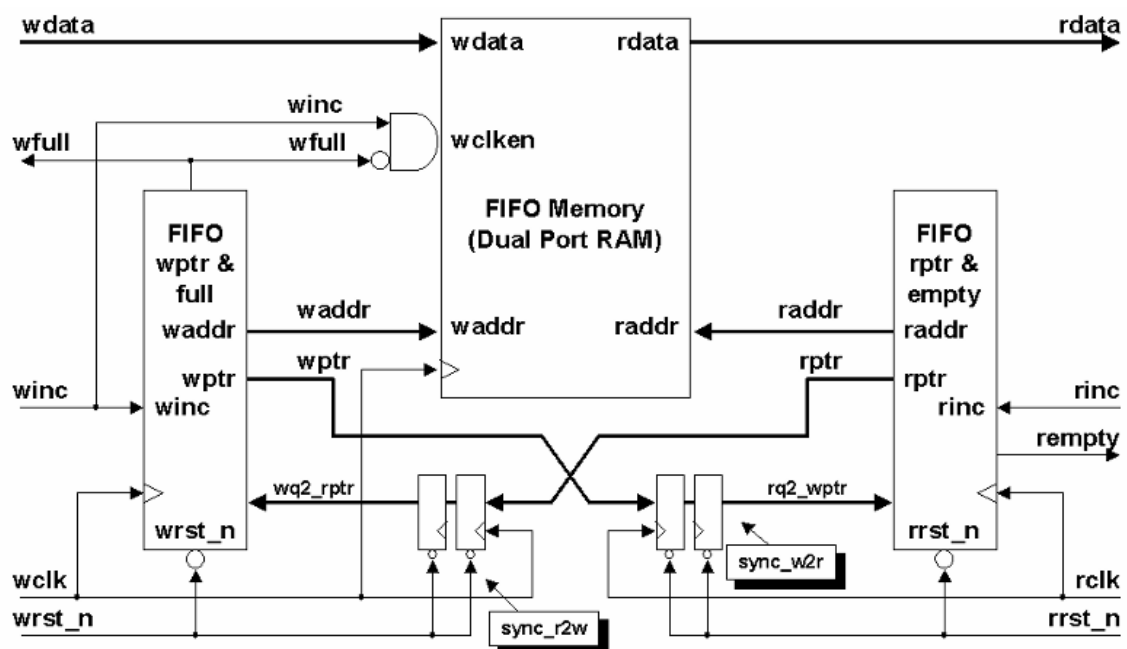
- Connects all submodules and routes signals between domains.
- Produces final outputs: write-full flag, read-empty flag, write data, and read data.
- Ensures correct coordination between pointer logic, synchronizers, and memory subsystem.

The overall design relies heavily on Gray-coded pointer arithmetic, synchronizer stability, and correct flag generation to function safely. These mechanisms allow the FIFO to operate efficiently under varying clock speeds and unpredictable timing conditions. The design must also correctly handle pointer wrap-around, maintain ordered data flow, and prevent invalid read/write actions under full or empty conditions. Because the behavior of asynchronous interfaces cannot be fully validated using traditional simulation, the architecture is an ideal candidate for assertion-based formal verification.

Block Diagram

The Async FIFO consists of five major modules:

1. **fifomem.sv**
Dual-port memory, written using wclk and read using rclk.
2. **wptr_full.sv**
 - Maintains binary & Gray write pointer
 - Handles full flag generation
 - Synchronizes read pointer (Gray) into write clock domain
3. **rptr_empty.sv**
 - Maintains binary & Gray read pointer
 - Handles empty flag generation
 - Synchronizes write pointer (Gray) into read clock domain
4. **sync_r2w.sv**
 - Two-flip synchronizer transferring rptr Gray into write domain
5. **sync_w2r.sv**
 - Two-flip synchronizer transferring wptr Gray into read domain



Signals and Description for each module.

1. `asynch_fifo_dut.sv` — Top-Level FIFO Wrapper

wclk	Input	Write clock domain; controls write pointer updates and memory writes.
rclk	Input	Read clock domain; controls read pointer updates and memory reads.
wrst_n	Input	Active-low reset for write domain logic.
rrst_n	Input	Active-low reset for read domain logic.
winc	Input	Write enable; requests a write into FIFO.
rinc	Input	Read enable; requests a read from FIFO.
wdata	Input	Data input for write operations.
rdata	Output	Data output from read operations.
wfull	Output	Indicates FIFO is full; write operations are blocked.
rempty	Output	Indicates FIFO is empty; read operations are blocked.
wptr	Internal	Gray-coded write pointer generated by write logic.
rptr	Internal	Gray-coded read pointer generated by read logic.
rq2_wptr	Internal	Synchronized write pointer in read domain.
wq2_rptr	Internal	Synchronized read pointer in write domain.

2. `wptr_full.sv` — Write Pointer and Full Logic

wclk	Input	Write domain clock.
wrst_n	Input	Active-low reset for write pointer logic.
winc	Input	Write request from environment.
wq2_rptr	Input	Read pointer synchronized into write domain.
wptr	Output	Gray-coded write pointer sent to synchronizer and memory.
wbin	Internal	Binary version of write pointer; used for addressing memory.
wfull	Output	Flag indicating FIFO is full.

wptr_next Internal Next Gray-coded write pointer value.

wbin_next Internal Next binary write pointer value.

Function: Maintains write pointer, converts binary → Gray code, evaluates full condition based on pointer distances and synchronized read pointer.

3. rptr_empty.sv — Read Pointer and Empty Logic

rclk Input Read domain clock.

rrst_n Input Active-low reset for read pointer logic.

rinc Input Read request from environment.

rq2_wptr Input Write pointer synchronized into read domain.

rptr Output Gray-coded read pointer sent to synchronizer and memory.

rbin Internal Binary version of read pointer.

empty Output FIFO empty status flag.

rptr_next Internal Next Gray-coded read pointer value.

rbin_next Internal Next binary read pointer value.

Function: Tracks read pointer, generates empty signal, and ensures no invalid reads occur when FIFO is empty.

4. sync_r2w.sv — Read Pointer Synchronizer (Read → Write Domain)

wclk Input Target domain clock (write domain).

wrst_n Input Active-low reset for synchronizer.

rptr Input Gray-coded read pointer from read domain.

wq1_rptr Internal First-stage synchronization flop.

wq2_rptr Output Stable synchronized read pointer in write domain.

Function: Prevents metastability by passing the read pointer through a 2-flip-flop synchronization pipeline.

5. sync_w2r.sv — Write Pointer Synchronizer (Write → Read Domain)

rclk	Input	Target domain clock (read domain).
rrst_n	Input	Active-low reset for synchronizer.
wptr	Input	Gray-coded write pointer from write domain.
rq1_wptr	Internal	First-stage synchronization register.
rq2_wptr	Output	Stable synchronized write pointer in read domain.

Function: Provides safe transfer of the write pointer into the read clock domain.

6. fifomem.sv — FIFO Memory

rdata	Output	Data output read from FIFO memory at the address raddr. Updated on rclk when read clock enable is active.
wdata	Input	Data to be written into the FIFO memory at address waddr on a valid write cycle.
waddr	Input	Binary write address generated from the write pointer. Selects memory location for storing wdata.
raddr	Input	Binary read address generated from the read pointer. Selects the memory location that drives rdata.
wclken	Input	Write clock enable; controls whether a write to memory occurs on the rising edge of wclk. Usually equals: wclken = winc && !wfull.
rclken	Input	Read clock enable; controls read access when rempty == 0. Enables reading from memory on rclk.
wfull	Input	Indicates FIFO is full; typically prevents writes when high.
rempty	Input	Indicates FIFO is empty; typically prevents reads when high.
wrst_n	Input	Active-low reset for the write-side memory interface. Clears write-side control logic.
rrst_n	Input	Active-low reset for the read-side memory interface. Clears read-side control logic.
wclk	Input	Write clock used to register write data into memory.
rclk	Input	Read clock used to register read data from memory.

Timing Diagrams:

- Write clock vs Read clock
- Write pointer update and Gray conversion
- Read pointer update
- Full flag timing
- Empty flag timing
- CDC synchronization latency (2-cycle delay)

Group	Signal	Cycle0	Cycle1	Cycle2	Cycle3	Cycle4	Cycle5	Cycle6	Cycle7
Input	wrst_n	0	1	1	1	1	1	1	1
Input	winc	0	1	1	1	1	1	1	1
Input	wdata	XXXX	0xAAA0	0xAAA0	0xBBB0	0xCCC0	XXXX	XXXX	XXXX
Output	wfull	0	0	0	0	0	0	0	0
Comment		Reset -> Empty	Write 1	Write 2	Write 3	IDLE	Read domain has started popping	Pointers equal + FIFO draining	
Input	rrst_n	0	1	1	1	1	1	1	1
Input	rinc	0	0	0	0	1	1	1	0
Input	rdata	XXXX	XXXX	XXXX	XXXX	0xAAA0	0xBBB0	0xCCC0	XXXX
Output	rempty	1	0	0	0	0	0	0	1
Comment		Reset released	No reads yet	No reads yet	No reads yet	Read 1	Read 2	Read 3	FIFO empty

Formal Verification Test Plan

We defined a structured FPV plan targeting four verification dimensions to ensure functional correctness, CDC safety, and architectural robustness of the Asynchronous FIFO.

1. Safety Properties

These ensure that *nothing illegal ever happens* in the FIFO.

- No write allowed when FIFO is full
- No read allowed when FIFO is empty
- Write pointer increments correctly
- Read pointer increments correctly
- Full/empty flags reflect pointer difference
- Gray code changes one bit per increment
- Synchronizer outputs (wq2_rptr, rq2_wprr) never jump backwards or glitch
- Binary and Gray-coded pointers remain consistent across all cycles
- Read and write addresses correctly map to lower bits of binary pointers

- No stale or invalid data is ever read from memory
- Memory locations are written and read in strict FIFO order
- No unintended pointer movement when enable signals (winc, rinc) are low

2. Liveness Properties

These ensure that *progress always occurs* if the environment allows.

- If FIFO not full → write will progress
- If FIFO not empty → read eventually returns data
- Full flag deasserts after enough reads
- Empty flag deasserts after enough writes
- Write pointer eventually wraps around after sufficient writes
- Read pointer eventually wraps around after sufficient reads
- Pointer synchronization eventually propagates updates across domains
- Design never deadlocks in a state where progress is possible
- Concurrent read/write scenarios eventually result in both pointers advancing

3. CDC Safety Properties

These ensure safe and deterministic behaviour across clock domains.

- Opposite-domain pointer sampled only at synchronizer outputs
- No combinational CDC paths
- No metastability propagation beyond first flop
- Synchronizer latency is exactly 2 cycles (no skipped or early sampling)
- Stable pointers in one domain eventually appear stable in the other
- Gray-code encoding ensures only single-bit changes cross domains
- No CDC-induced corruption of full/empty logic
- Both synchronizers correctly reset to known values in their respective domains

4. Reset Integrity Properties

These ensure the FIFO starts in a known good state.

- All pointers start at zero

- rempty initializes to 1
- wfull initializes to 0
- Direction bit resets correctly
- Synchronizer registers reset to 0 in both domains
- Memory output returns to a known idle value on reset
- No pointer movement allowed until reset is fully deasserted
- Reset release is monotonic (no reset glitches)
- Full/empty flags must stabilize within a bounded number of cycles after reset

Explanation of Assumptions Used

- No X/Z on critical signals

assume property (!\$isunknown(winc));

assume property (!\$isunknown(rinc));

assume property (!\$isunknown(rrst_n) && !\$isunknown(wrst_n));

assume property (!\$isunknown(rclk));

assume property (!\$isunknown(wclk));

Explanation: We assume clocks, resets, and control signals are never X/Z, so the formal tool only analyzes legal 0/1 behavior and not undefined/glitchy states.

- Clocks are free-running

assume property (@(posedge wclk) \$fell(wclk));

assume property (@(negedge wclk) \$rose(wclk));

assume property (@(posedge rclk) \$fell(rclk));

assume property (@(negedge rclk) \$rose(rclk));

Explanation: We assume wclk and rclk keep toggling and don't get stuck, modeling continuous operation of the FIFO instead of trivial "clock stopped" cases.

- Resets are clean and don't glitch

assume property (@(posedge wclk) \$rose(wrst_n) |-> \$stable(wrst_n));

assume property (@(posedge rclk) \$rose(rrst_n) |-> \$stable(rrst_n));

Explanation: Once reset is deasserted, it must stay stable; this models a clean reset release and avoids spurious reset pulses in formal.

- Enables are stable within a cycle (winc, rinc)

assume property (@(posedge wclk) \$stable(winc));

assume property (@(posedge rclk) \$stable(rinc));

Explanation: Write and read enables don't glitch multiple times in one clock period, matching realistic handshake behaviour.

- Write data stable when writing

assume property (@(posedge wclk) winc |-> \$stable(wdata));

Explanation: When a write occurs, wdata must be stable for that cycle, modelling valid, setup-and-hold-clean data at the FIFO input.

- No write when full

assume property (wfull |-> !winc);

Explanation: The environment respects back-pressure: it never writes when the FIFO is full, so we only verify legal usage.

- No read when empty

assume property (empty |-> !rinc);

Explanation: The environment never reads when the FIFO is empty, preventing illegal underflow scenarios in the formal search space.

Explanation of Assertions Used

We wrote properties for all modules:

1. Synchronizer – Read Pointer into Write Domain (sync_r2w)

- WQ_RPTR_RESET

Checks: On write-domain reset, both synchronizer flops for the read pointer (wq1_rptr, wq2_rptr) are cleared to 0.

Guarantees a clean, known starting state for CDC logic in the write clock domain.

- WQ1_RPTR_LATENCY

Checks: wq1_rptr captures the read pointer (rptr) with one write-clock cycle of latency.

Ensures the first synchronizer stage really behaves like a 1-cycle sampled copy of the source pointer.

- WQ2_RPTR_LATENCY

Checks: wq2_rptr captures wq1_rptr with one additional write-clock cycle of latency.

Confirms the classic two-flop synchronizer structure and that the second stage follows the first correctly.

- WQ2_RPTR_MATCH_STABLE_RPTR

Checks: If rptr stays stable for at least two cycles, then two cycles later wq2_rptr matches that value.

Proves that a stable read pointer will eventually be seen correctly and stably in the write domain.

2. Synchronizer – Write Pointer into Read Domain (sync_w2r)

- RQ_WPTR_RESET

Checks: On read-domain reset, both synchronizer flops for the write pointer (rq1_wptr, rq2_wptr) are cleared.

Guarantees a consistent, empty starting state from the read side's perspective.

- RQ1_WPTR_LATENCY & RQ2_WPTR_LATENCY

Checks: RQ1_WPTR_LATENCY: rq1_wptr captures the write pointer in one read-clock step,

RQ2_WPTR_LATENCY: rq2_wptr captures rq1_wptr in the next read-clock step.

Formally confirms the two-stage synchronization of the write pointer into the read domain.

- RQ2_MATCH_STABLE_WPTR

Checks: If wptr remains stable for two cycles, then two cycles later rq2_wptr equals that earlier value.

Shows that any stable write pointer will eventually propagate correctly to the read side.

3. Write Pointer, Addressing & Full Flag (wptr_full)

- WBIN_WPTR_RESET

Checks: After write-domain reset, wbin, wptr, and wfull are all cleared.

Ensures that the FIFO starts in a “not full, pointer at 0” state on the write side.

- WADDR_BIN_MAPPING

Checks: waddr equals the lower ADDR_WIDTH bits of the binary write pointer wbin.

Proves that the write address used for memory access is correctly derived from the internal pointer.

- WPTR_GRAY_MAPPING

Checks: The Gray-coded write pointer (wptr) matches the Gray conversion of the next binary pointer (wbinnext).

Ensures correct Gray encoding, which is critical for safe CDC of pointers.

- WBIN_HOLD_WHEN_NO_WINC

Checks: When winc is low, wbin does not change.

Confirms that the write pointer only advances on a valid write request.

- WFULL_MATCHES_EXPR

Checks: The registered full flag wfull equals the internal expression wfull_val.
Proves that the exposed full flag correctly reflects the internal full-detection logic.

4. Read Pointer, Addressing & Empty Flag (rptr_empty)

- R_BIN_RPTR_EMPTY_RESET

Checks: After read-domain reset, rbin and rptr go to 0 and rempty goes to 1.
Guarantees the FIFO is seen as empty after reset from the read side.

- RADDR_BIN_MAPPING

Checks: raddr equals the lower bits of rbin.
Ensures the read address tracks the binary read pointer correctly.

- RPTR_GRAY_MAPPING

Checks: rptr equals the Gray-coded version of the next binary read pointer (rbinnext).
Confirms correct Gray encoding for the read pointer before synchronizing it.

- R_BIN_INCREMENT_WHEN_ENABLED

Checks: When rinc is high and FIFO is not empty, rbin and rptr update to the next values.
Ensures the read pointer only advances on legal read operations and that Gray and binary views are consistent.

- R_BIN_HOLD_WHEN_NO_RINC

Checks: When rinc is low, rbin holds its previous value.
Confirms that there is no unintended pointer movement without a read request.

- REMPTY_MATCHES_EXPR & REMPTY_VAL_EXPR

Checks: REMPTY_VAL_EXPR: combinational empty expression rempty_val matches rgraynext == rq2_wptr. REMPTY_MATCHES_EXPR: the registered rempty flag tracks rempty_val.

5. Data Integrity & Read Behaviour (fifomem)

- READ_RETURNS_FIFO_DATA

Checks: When a read is enabled (rclken) and FIFO is not empty, rdata equals the stored FIFO memory at address raddr (from the previous cycle).

Why important: This is your core data integrity assertion – it proves that the data read from the FIFO is exactly what was stored at that location.

- READ_RETURNS_ZERO_WHEN_IDLE

Checks: When there is no active read (!rclken) or FIFO is empty, rdata is driven to 0.

Why important: Defines idle behaviour at the output and avoids leaking stale data when no valid read occurs.

Explanation of Cover Properties Used

- FIFO_NON_EMPTY – FIFO becomes non-empty

Explanation: Covers a scenario where the FIFO starts empty (rempty = 1) and later becomes non-empty (rempty = 0). This shows that writes are actually happening and the FIFO can hold valid data.

- FIFO_FULL – FIFO reaches full condition

Explanation: Covers any trace where wfull goes high at least once. This confirms that the formal tool explored deep enough write activity to actually fill the FIFO.

- FIFO_FULL_TO_NOTFULL – Full → Not-full transition

Explanation: Covers a sequence where the FIFO becomes full and later returns to not-full. This demonstrates proper recovery from a full condition as reads drain entries.

- FIFO_EMPTY_NONEMPTY_EMPTY – Empty → Non-empty → Empty

Explanation: Covers a complete cycle: FIFO starts empty, receives data (non-empty), then is read back to empty again.

This shows end-to-end FIFO usage: fill, drain, and correct empty flag behavior.

- WPTR_WRAP – Write pointer wrap-around

Explanation: Covers the event where the binary write pointer goes from the last address (DEPTH-1) back to 0. This confirms correct wrap-around behavior of the write pointer.

- RPTR_WRAP – Read pointer wrap-around

Explanation: Covers the read pointer wrapping from the last location back to 0. This shows that the read side can traverse the entire FIFO depth without errors.

Analysis / Waveforms

- Correct write pointer Gray progression
- Read pointer Gray progression
- Full flag logic hitting preset when pointers match with direction=1
- Empty flag logic hitting preset when pointers match with direction=0
- CDC 2-flop synchronizers introducing expected 2-cycle latency

- No X-propagation in functional mode
- Proper read-after-write behaviour across clocks

Bug Injection and Debugging:

FPV Failures:

Bug Found 1:

```

22
23 // Memory read-address pointer (okay to use binary to address memory)
24 assign raddr = rbin[ADDR_WIDTH-1:0];
25 assign rbinnext = (!rrst_n) ? 0 : (rbin + (rinc & ~rempty));
26 assign rgraynext = (!rrst_n) ? 0 : ((rbinnext>>1) ^ ~rbinnext);
27

```

status (V)	depth	name	vacuity	witness
✗ 2	2	async_fifo_dut.assertion_h.RPTR_GRAY_MAPPING_A	✓ 1	
✗ 4	4	async_fifo_dut.assertion_h.WPTR_GRAY_SINGLE_BIT_CHANGE_A	✓ 1	
✗ 3	3	async_fifo_dut.assertion_h.WQ1_RPTR_LATENCY_A	✓ 1	
✗ 7	7	async_fifo_dut.assertion_h.WQ2_MATCH_STABLE_RPTR_A	✓ 4	

Task	Property Summary	Assertion	Cover	Constraint	Vacuity	Witness
FPV	Number of Properties	27	0	2	18	0
	Proven (P) / Covered (C)	23	0	-	18	0
	Failed (F) / Uncoverable (U)	4	0	-	0	0
	Vacuous (V)	0	-	-	-	-
	Inconclusive (I)	0	-	-	0	0
	Checking (CH)	0	-	-	0	0

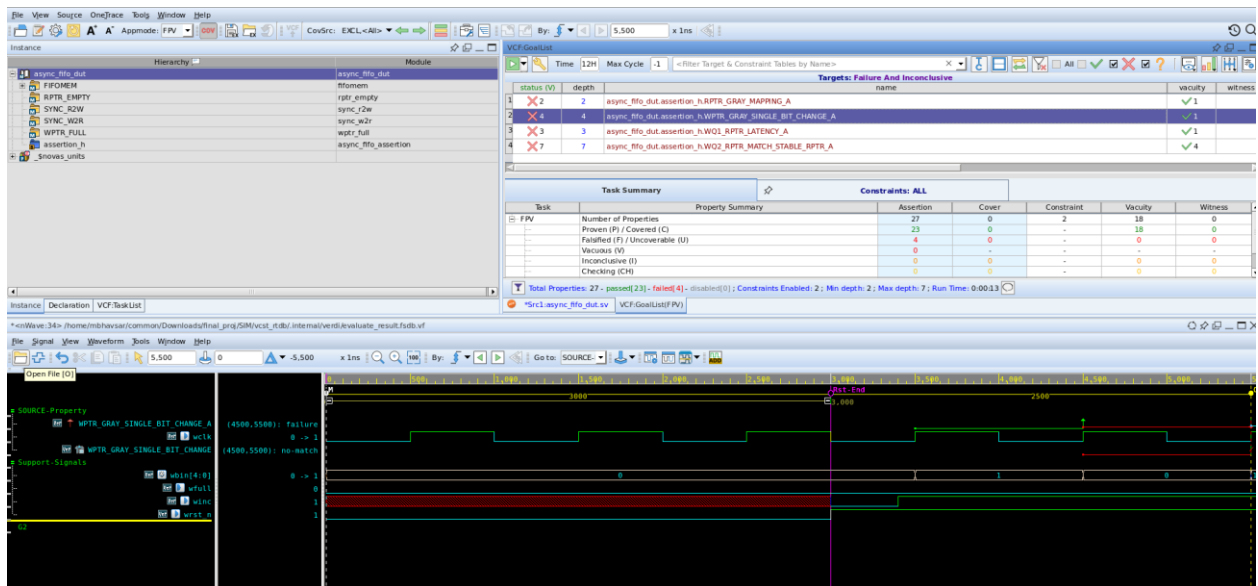
Total Properties: 27 - passed(23) - failed(4) - disabled(0) - Constraints Enabled: 2; Mn depth: 2; Max depth: 7; Run Time: 0:00:13

Bug Found 2:

```

7 //Procedural blocks
8 always_ff@(posedge wclk or negedge wrst_n)
9 begin
10 if(!wrst_n)
11 begin
12 {wq2_rptr, wq1_rptr} <= 0;
13 end
14 else
15 begin
16 {wq2_rptr, wq1_rptr} <= {wq1_rptr, rptr << 1};
17 end
18 end
19

```

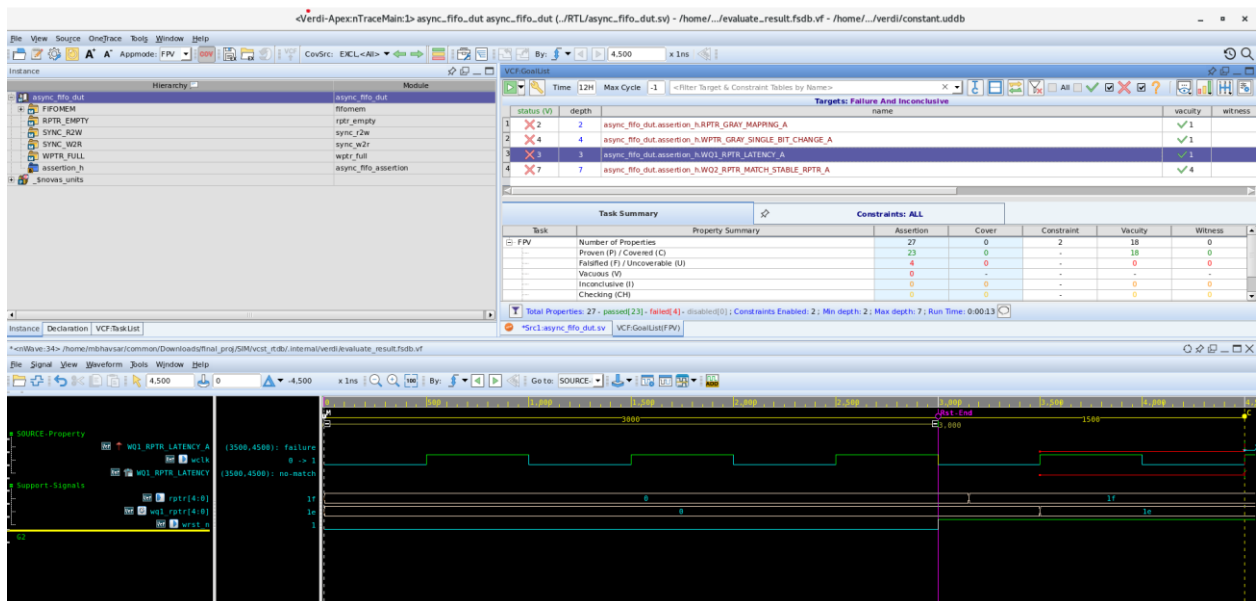


Bug Found 3:

```

21 // Memory write-address pointer (okay to use binary to address memory)
22 assign waddr = wbin[ADDR_WIDTH-1:0];
23 assign wbinnext = (!wrst_n) ? 0 : (wbin ^ (winc & ~wfull));
24 assign wgraynext = (!wrst_n) ? 0 : ((wbinnext>>1) ^ wbinnext);
25 //

```



Resolved DUT bugs as below:

Solution for 1:

```

23 // Memory read-address pointer (okay to use binary to address memory)
24 assign raddr = rbin[ADDR_WIDTH-1:0];
25 assign rbinnext = (!rrst_n) ? 0 : (rbin + (rinc & ~rempty));
26 assign rgraynext = (!rrst_n) ? 0 : ((rbinnext>>1) ^ rbinnext);
27 //

```

Solution for 2:

```
7    //Procedural blocks
8    always_ff@(posedge wclk or negedge wrst_n)
9    begin
10       if(!wrst_n)
11       begin
12          {wq2_rptr, wq1_rptr} <= 0;
13       end
14       else
15       begin
16          {wq2_rptr, wq1_rptr} <= {wq1_rptr, rptr};
17       end
18    end
```

Solution for 3:

```
21    // Memory write-address pointer (okay to use binary to address memory)
22    assign waddr = wbin[ADDR_WIDTH-1:0];
23    assign wbinnext = (!wrst_n) ? 0 : (wbin + (winc & ~wfull));
24    assign wgraynext = (!wrst_n) ? 0 : ((wbinnext>>1) ^ wbinnext);
25    //
```

The screenshot shows the VCS-Postlist tool interface. On the left, a 'Task List' pane shows the project 'FPU' with a progress bar at 27:07:10.4. The main pane displays a table of assertions with columns for status, depth, name, vacuity, and width.

status	depth	name	vacuity	width
✓	1	async_fifo_dut.assertion_n.READOR_BIN_MAPPING_A		
✓	2	async_fifo_dut.assertion_n.READOR_RESET_A		
✓	3	async_fifo_dut.assertion_n.READ_RETURNS_FIFO_DATA_A	✓5	
✓	4	async_fifo_dut.assertion_n.READ_RETURNS_ZERO_WHEN_IDLE_A	✓1	
✓	5	async_fifo_dut.assertion_n.REEMPTY_MATCHES_EXPR_A	✓1	
✓	6	async_fifo_dut.assertion_n.REEMPTY_VAL_EXPR_A		
✓	7	async_fifo_dut.assertion_n.RPTR_GRAY_MAPPING_A	✓1	
✓	8	async_fifo_dut.assertion_n.RQ2_WPTR_EDGE_CHANGE_A	✓3	
✓	9	async_fifo_dut.assertion_n.RQ2_WPTR_LATENCY_A	✓1	
✓	10	async_fifo_dut.assertion_n.RQ2_MATCH_STABLE_WPTR_A	✓1	
✓	11	async_fifo_dut.assertion_n.RQ2_WPTR_EDGE_CHANGE_A	✓4	
✓	12	async_fifo_dut.assertion_n.RQ2_WPTR_LATENCY_A	✓1	
✓	13	async_fifo_dut.assertion_n.RQ2_WPTR_RESET_A		
✓	14	async_fifo_dut.assertion_n.RR_BIN_HOLD_WHEN_NO_BIN_A	✓1	
✓	15	async_fifo_dut.assertion_n.RR_BIN_INCREMENT_WHEN_ENABLED_A	✓5	
✓	16	async_fifo_dut.assertion_n.RR_EMPTY_RESET_A		
✓	17	async_fifo_dut.assertion_n.WADDR_BIN_MAPPING_A		
✓	18	async_fifo_dut.assertion_n.WADDR_RESET_A		
✓	19	async_fifo_dut.assertion_n.WBIN_HOLD_WHEN_NO_WINC_A	✓1	
✓	20	async_fifo_dut.assertion_n.WBIN_WPTR_RESET_A		
✓	21	async_fifo_dut.assertion_n.WFULL_MATCHES_EXPR_A	✓1	
✓	22	async_fifo_dut.assertion_n.WPTR_GRAY_MAPPING_A	✓1	
✓	23	async_fifo_dut.assertion_n.WPTR_GRAY_SINGLE_BIT_CHANGE_A	✓1	
✓	24	async_fifo_dut.assertion_n.WQ1_RPTR_LATENCY_A	✓1	

Challenges Faced

- Understanding Gray-code quadrant logic for early full/empty detection
- Debugging direction-bit behaviour (set/clear logic)
- Managing metastability at CDC boundaries
- Formal convergence difficulty due to deep pointer state space
- Ensuring assumptions do not over-constrain design
- Jasper Gold vs VC Formal behaviour differences in solving AEP failures
- Handling long-depth FIFO pointer proofs requiring induction

Execution of different Formal Application:

1) FPV Application:

TCL File (Possibility 1: Both wclk and rclk are equal):

```
fca_run.tcl  fpv_run.tcl  fxp_run.tcl
1 set_fml_appmode FPV
2 set design async_fifo_dut
3
4 read_file -top $design -format sverilog -sva -aep all -vcs {-f ../RTL/filelist}
5
6 # Creating clock and reset signals
7 create_clock wclk -period 100 -initial 0
8 create_reset wrst_n -sense low
9
10 # Creating clock and reset signals
11 create_clock rclk -period 100 -initial 0
12 create_reset rrst_n -sense low
13
14 # Runing a reset simulation
15 sim_run -stable
16 sim_save_reset
17
18 check fv
19 report_fv
```

TCL File (Possibility 2: wclk is faster than rclk):

```
fpv_run.tcl (/home/mbhavsar/common/Downloads/final_proj/SIM) (2 of
File Edit Tools Syntax Buffers Window Help
fca_run.tcl  fpv_run.tcl  fxp_run.tcl
1 set_fml_appmode FPV
2 set design async_fifo_dut
3
4 read_file -top $design -format sverilog -sva -aep all -vcs {-f ../RTL/filelist}
5
6 # Creating clock and reset signals
7 create_clock wclk -period 70 -initial 0
8 create_reset wrst_n -sense low
9
10 # Creating clock and reset signals
11 create_clock rclk -period 100 -initial 0
12 create_reset rrst_n -sense low
13
14 # Runing a reset simulation
15 sim_run -stable
16 sim_save_reset
17
18 check fv
19 report_fv
```

TCL File (Possibility 3: rclk is faster than wclk):

```

fca_run.tcl + fpv_run.tcl fxp_run.tcl
1 set_fml_appmode FPV
2 set design async_fifo_dut
3
4 read_file -top $design -format sverilog -sva -aep all -vcs {-f ../RTL/filelist}
5
6 # Creating clock and reset signals
7 create_clock wclk -period 100 -initial 0
8 create_reset wrst_n -sense low
9
10 # Creating clock and reset signals
11 create_clock rclk -period 70 -initial 0
12 create_reset rrst_n -sense low
13
14 # Running a reset simulation
15 sim_run -stable
16 sim_save_reset
17
18 check fv
19 report_fv

```

Created async_fifo_assertion, async_fifo_assumption, and async_fifo_cover files to verify the DUT features through formal verification methods using VC formal tool. Initially, there are multiple failures of below properties and then I found some clock delay issue and modify the properties as well.

status (V)	depth	name	vacuity	witness	type	engine	elapsed_time
1	0	async_fifo_dut.assertion.h.EMPTY_AFTER_GLOBAL_RESET_A	✗		assert	..URALLY	
2	0	async_fifo_dut.assertion.h.FULL_AND_EMPTY_NEVER_SIMULTANEOUS_A	✗		assert	rt1	00:00:02
3	0	async_fifo_dut.assertion.h.RADDR_RESET_A	✓0		assert	rt1	00:00:01
4	0	async_fifo_dut.assertion.h.READ_RETURNS_FIFO_DATA_A	✓0		assert	rt1	00:00:01
5	0	async_fifo_dut.assertion.h.READ_RETURNS_ZERO_WHEN_IDLE_A	✓0		assert	rt1	00:00:01
6	0	async_fifo_dut.assertion.h.REEMPTY_MATCHES_EXPR_A			assert	rt1	00:00:01
7	0	async_fifo_dut.assertion.h.RPTR_GRAY_MAPPING_A			assert	rt1	00:00:01
8	0	async_fifo_dut.assertion.h.RPTR_GRAY_SINGLE_BIT_CHANGE_A	✓0		assert	rt1	00:00:01
9	0	async_fifo_dut.assertion.h.RQ1_WPTR_EDGE_CHANGE_A			assert	rt1	00:00:01
10	0	async_fifo_dut.assertion.h.RQ1_WPTR_LATENCY_A			assert	rt1	00:00:01
11	0	async_fifo_dut.assertion.h.RQ2_WPTR_EDGE_CHANGE_A			assert	rt1	00:00:01
12	0	async_fifo_dut.assertion.h.RQ2_WPTR_LATENCY_A			assert	rt1	00:00:01
13	0	async_fifo_dut.assertion.h.RQ_WPTR_RESET_A	✓0		assert	rt1	00:00:01
14	2	async_fifo_dut.assertion.h.R_BIN_HOLD_WHEN_EMPTY_A	✓0		assert	rt1	00:00:01
15	0	async_fifo_dut.assertion.h.R_BIN_HOLD_WHEN_NO_RINC_A	✓0		assert	rt1	00:00:01
16	0	async_fifo_dut.assertion.h.R_BIN_INCREMENT_WHEN_ENABLED_A	✓0		assert	rt1	00:00:01

I have attached few cases where the properties became fail. It includes the older property definition, failure traces and updated property with passing condition.

Case:1

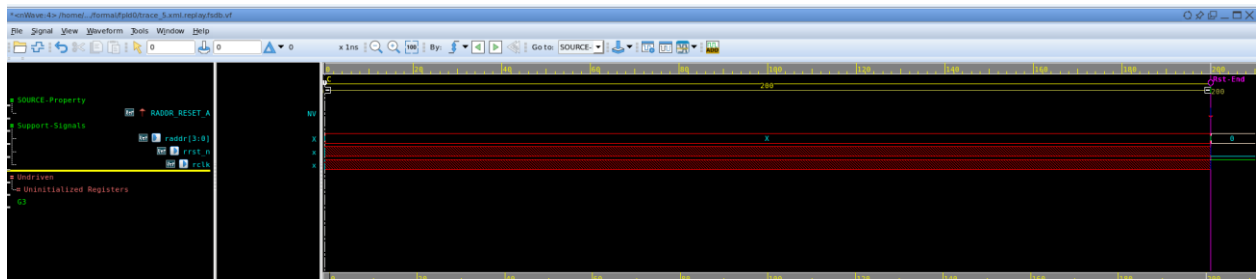
Failed Property:

```

// raddr reset
property RADDR RESET;
    @(posedge rclk)
    !rrst_n |-> (async_fifo_dut.RPTR_EMPTY.raddr=='0');
endproperty
RADDR RESET A: assert property(RADDR RESET);

```

Failed Trace:



Updated Property:

```

192 // raddr reset
193 property RADDR_RESET;
194   @(posedge rclk)
195   !rrst_n | => (async_fifo_dut.RPTR_EMPTY.raddr=='0');
196 endproperty
197 RADDR_RESET_A: assert property(RADDR_RESET);
198

```

Case:2

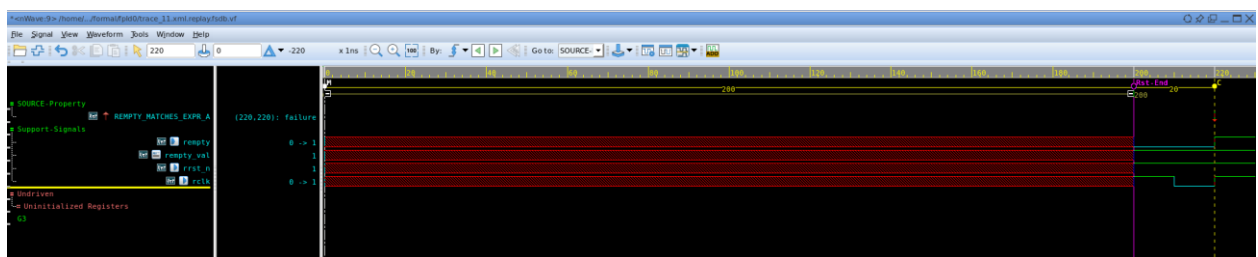
Failed Property:

```

241 // empty correctness
242 property REMPTY_MATCHES_EXPR;
243   @(posedge rclk)
244   rrst_n | => async_fifo_dut.RPTR_EMPTY.empty == async_fifo_dut.RPTR_EMPTY.empty_val;
245 endproperty
246 REMPTY_MATCHES_EXPR_A: assert property(REMPY_MATCHES_EXPR);
247

```

Failed Trace:



Updated Property:

```

241 // empty correctness
242 property REMPTY_MATCHES_EXPR;
243   @(posedge rclk) disable iff (!rrst_n)
244   (rrst_n == 1) | => (async_fifo_dut.RPTR_EMPTY.empty == $past(async_fifo_dut.RPTR_EMPTY.empty_val));
245 endproperty
246 REMPTY_MATCHES_EXPR_A: assert property(REMPY_MATCHES_EXPR);
247

```

Case:3

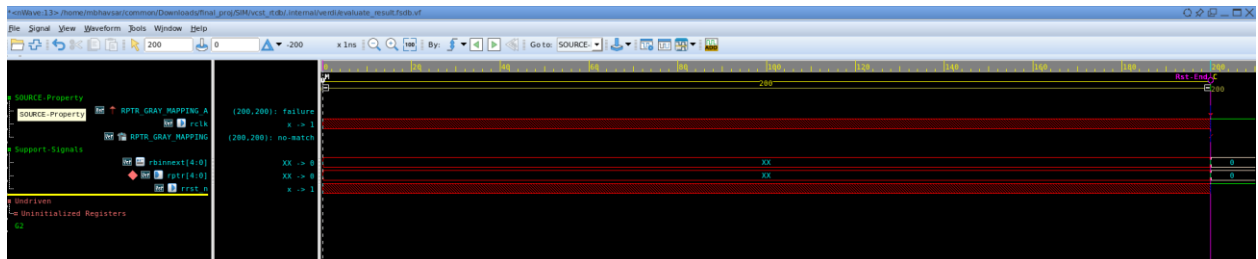
Failed Property:

```

6 // rptr gray encoding
7 property RPTR_GRAY_MAPPING;
8   @(posedge rclk) disable iff (!rrst n)
9   [ async_fifo_dut.RPTR_EMPTY.rptr == (async_fifo_dut.RPTR_EMPTY.rbinnext ^ (async_fifo_dut.RPTR_EMPTY.rbinnext>>1));
0 endproperty
1 RPTR_GRAY_MAPPING_A: assert property(RPTR_GRAY_MAPPING);
2

```

Failed Trace:



Updated Property:

```

6 // rptr gray encoding
7 property RPTR_GRAY_MAPPING;
8   @(posedge rclk) disable iff (!rrst n)
9   rrst_n | => async_fifo_dut.RPTR_EMPTY.rptr == $past(async_fifo_dut.RPTR_EMPTY.rbinnext ^ (async_fifo_dut.RPTR_EMPTY.rbinnext>>1));
0 endproperty
1 RPTR_GRAY_MAPPING_A: assert property(RPTR_GRAY_MAPPING);
2

```

Case:4

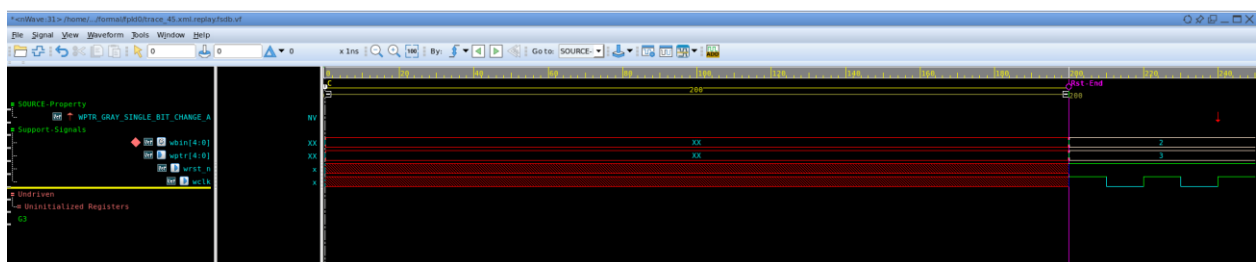
Failed Property:

```

42 // Gray code single bit transition
43 property WPTR_GRAY_SINGLE_BIT_CHANGE;
44   @(posedge wclk) disable iff (!wrst n)
45   wrst_n | => (async_fifo_dut.WPTR_FULL.wbin == $past(async_fifo_dut.WPTR_FULL.wbin)+1) | => $onehot(async_fifo_dut.WPTR_FULL.wptr ^ $past(async_fifo_dut.WPTR_FULL.wptr));
46 endproperty
47 WPTR_GRAY_SINGLE_BIT_CHANGE_A: assert property(WPTR_GRAY_SINGLE_BIT_CHANGE);
48

```

Failed Trace:



Updated Property:

```

42 // Gray code single bit transition
43 property WPTR_GRAY_SINGLE_BIT_CHANGE;
44   @(posedge wclk) disable iff (!wrst n)
45   wrst_n | => (async_fifo_dut.WPTR_FULL.wbin == $past(async_fifo_dut.WPTR_FULL.wbin + (async_fifo_dut.WPTR_FULL.winc & ~async_fifo_dut.WPTR_FULL.wfull)));
46 endproperty
47 WPTR_GRAY_SINGLE_BIT_CHANGE_A: assert property(WPTR_GRAY_SINGLE_BIT_CHANGE);
48

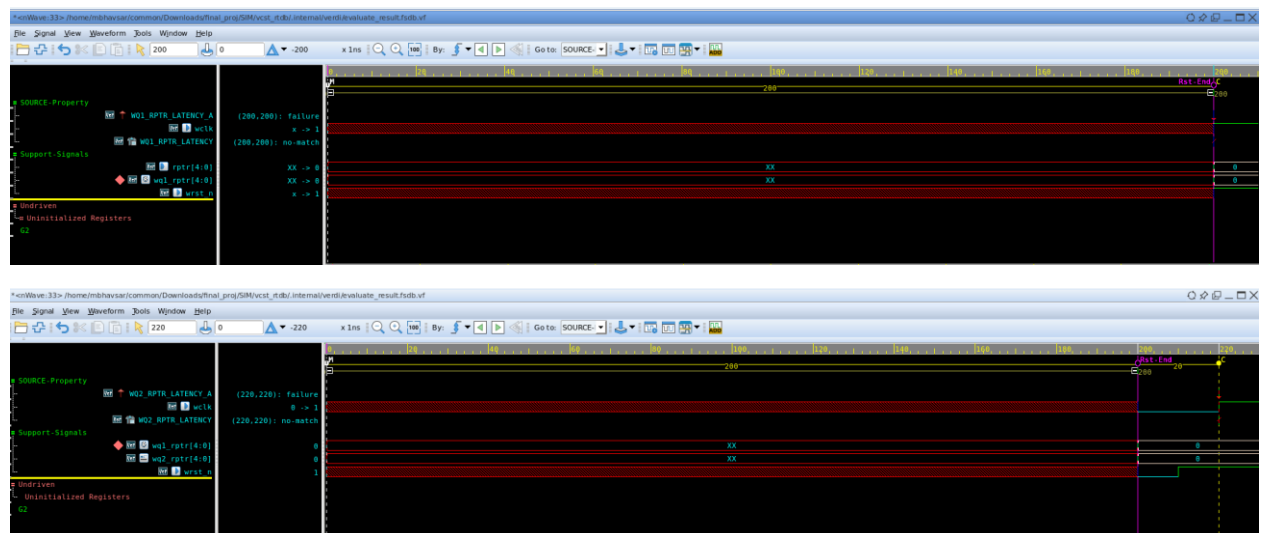
```

Case:5

Failed Property:

```
2
3 // wq1_rptr captures rprr with 1-cycle latency
4 property WQ1_RPTR_LATENCY;
5     @(posedge wclk) disable iff (!wrst_n)
6         async_fifo_dut.SYNC_R2W.wq1_rptr == $past(async_fifo_dut.RPTR_EMPTY.rptr);
7 endproperty
8 WQ1_RPTR_LATENCY_A: assert property(WQ1_RPTR_LATENCY);
9
10 // wq2_rptr captures wq1_rptr with 1-cycle latency
11 property WQ2_RPTR_LATENCY;
12     @(posedge wclk) disable iff (!wrst_n)
13         async_fifo_dut.wq2_rptr == $past(async_fifo_dut.SYNC_R2W.wq1_rptr);
14 endproperty
15 WQ2_RPTR_LATENCY_A: assert property(WQ2_RPTR_LATENCY);
16
```

Failed Traces:



Updated Property:

```
13 // wq1_rptr captures rprr with 1-cycle latency
14 property WQ1_RPTR_LATENCY;
15     @(posedge wclk) disable iff (!wrst_n)
16         wrst_n | => async_fifo_dut.SYNC_R2W.wq1_rptr == $past(async_fifo_dut.RPTR_EMPTY.rptr);
17 endproperty
18 WQ1_RPTR_LATENCY_A: assert property(WQ1_RPTR_LATENCY);
19
20 // wq2_rptr captures wq1_rptr with 1-cycle latency
21 property WQ2_RPTR_LATENCY;
22     @(posedge wclk) disable iff (!wrst_n)
23         wrst_n | => async_fifo_dut.wq2_rptr == $past(async_fifo_dut.SYNC_R2W.wq1_rptr);
24 endproperty
25 WQ2_RPTR_LATENCY_A: assert property(WQ2_RPTR_LATENCY);
26
```

- This kind of similar issues found on all properties which was solved later and then all properties became pass/proven. Also, some unnecessary or duplicate properties have been removed and added some meaningful into the assertion file.

Successful FPV result in VC Formal (Same passing result for all 3 possibilities of wclk and rclk clock speed):

Name	Progress	Result
FPV	27/27 (100%)	Passed

status	depth	name	vacuity	witness
✓	1	async_fifo_dut.assertion_h_RADDR_BIN_MAPPING_A		
✓	2	async_fifo_dut.assertion_h_RADDR_RESET_A	✓ 1	
✓	3	async_fifo_dut.assertion_h_READ_RETURNS_FIFO_DATA_A	✓ 2	
✓	4	async_fifo_dut.assertion_h_READ_RETURNS_ZERO_WHEN_IDLE_A	✓ 2	
✓	5	async_fifo_dut.assertion_h_REEMPTY_MATCHES_EXPR_A	✓ 2	
✓	6	async_fifo_dut.assertion_h_REEMPTY_VAL_EXPR_A		
✓	7	async_fifo_dut.assertion_h_RPTR_GRAY_MAPPING_A	✓ 2	
✓	8	async_fifo_dut.assertion_h_RQ2_WPTR_EDGE_CHANGE_A	✓ 0	
✓	9	async_fifo_dut.assertion_h_RQ2_WPTR_LATENCY_A	✓ 2	
✓	10	async_fifo_dut.assertion_h_RQ2_MATCH_STABLE_WPTR_A	✓ 0	
✓	11	async_fifo_dut.assertion_h_RQ2_WPTR_EDGE_CHANGE_A	✓ 2	

Total Properties: 27 - passed(27) - failed(0) - disabled(0) | Constraints Enabled: 0 | Run Time: 0:00:07

status	depth	name	vacuity	witness	type	engine	elapsed time
✓	1	async_fifo_dut.assertion_h_RADDR_BIN_MAPPING_A			assert	rq1	00:00:00
✓	2	async_fifo_dut.assertion_h_RADDR_RESET_A	✓ 1		assert	rq1	00:00:00
✓	3	async_fifo_dut.assertion_h_READ_RETURNS_FIFO_DATA_A	✓ 2		assert	rq1	00:00:00
✓	4	async_fifo_dut.assertion_h_READ_RETURNS_ZERO_WHEN_IDLE_A	✓ 2		assert	rq1	00:00:00
✓	5	async_fifo_dut.assertion_h_REEMPTY_MATCHES_EXPR_A	✓ 2		assert	rq1	00:00:00
✓	6	async_fifo_dut.assertion_h_REEMPTY_VAL_EXPR_A			assert	rq1	00:00:00
✓	7	async_fifo_dut.assertion_h_RPTR_GRAY_MAPPING_A	✓ 2		assert	rq1	00:00:00
✓	8	async_fifo_dut.assertion_h_RQ2_WPTR_EDGE_CHANGE_A	✓ 0		assert	rq1	00:00:00
✓	9	async_fifo_dut.assertion_h_RQ2_WPTR_LATENCY_A	✓ 2		assert	rq1	00:00:00
✓	10	async_fifo_dut.assertion_h_RQ2_MATCH_STABLE_WPTR_A	✓ 0		assert	rq1	00:00:00
✓	11	async_fifo_dut.assertion_h_RQ2_WPTR_EDGE_CHANGE_A	✓ 2		assert	rq1	00:00:00
✓	12	async_fifo_dut.assertion_h_RQ2_WPTR_LATENCY_A	✓ 2		assert	rq1	00:00:01
✓	13	async_fifo_dut.assertion_h_RQ_WPTR_RESET_A	✓ 1		assert	rq1	00:00:01
✓	14	async_fifo_dut.assertion_h_R_BIN_HOLD_WHEN_NO_RINC_A	✓ 1		assert	rq1	00:00:01
✓	15	async_fifo_dut.assertion_h_R_BIN_INCREMENT_WHEN_ENABLED_A	✓ 2		assert	rq1	00:00:01
✓	16	async_fifo_dut.assertion_h_R_BIN_RPTR_EMPTY_RESET_A	✓ 1		assert	rq1	00:00:01
✓	17	async_fifo_dut.assertion_h_WADDR_BIN_MAPPING_A			assert	rq1	00:00:01
✓	18	async_fifo_dut.assertion_h_WADDR_RESET_A	✓ 0		assert	rq1	00:00:01
✓	19	async_fifo_dut.assertion_h_WBIN_HOLD_WHEN_NO_WINC_A	✓ 2		assert	rq1	00:00:01
✓	20	async_fifo_dut.assertion_h_WBIN_WPTR_RESET_A	✓ 0		assert	rq1	00:00:01
✓	21	async_fifo_dut.assertion_h_WFULL_MATCHES_EXPR_A	✓ 0		assert	rq1	00:00:01
✓	22	async_fifo_dut.assertion_h_WPTR_GRAY_MAPPING_A	✓ 0		assert	rq1	00:00:01
✓	23	async_fifo_dut.assertion_h_WPTR_GRAY_SINGLE_BIT_CHANGE_A	✓ 2		assert	rq1	00:00:01
✓	24	async_fifo_dut.assertion_h_WQ1_RPTR_LATENCY_A	✓ 0		assert	rq1	00:00:01
✓	25	async_fifo_dut.assertion_h_WQ2_RPTR_LATENCY_A	✓ 2		assert	rq1	00:00:01
✓	26	async_fifo_dut.assertion_h_WQ2_RPTR_MATCH_STABLE_RPTR_A	✓ 4		assert	rq1	00:00:01
✓	27	async_fifo_dut.assertion_h_WQ_RPTR_RESET_A	✓ 0		assert	rq1	00:00:01

2) FXP Application:

TCL File (Possibility 1: Both wclk and rclk are equal):

```
fca_run.tcl  fpv_run.tcl  fxp_run.tcl

1 set_fml_appmode FXP
2
3 set design async_fifo_dut
4
5 read_file -top $design -format sverilog -sva -vcs {-f ../RTL/filelist}
6
7 create_clock rclk -period 100
8 create_reset rrst_n -sense low
9
10 create_clock wclk -period 100
11 create_reset wrst_n -sense low
12
13 sim_run -stable
14 sim_save_reset
15
16 fxp_generate
```

TCL File (Possibility 2: rclk is faster than wclk):

```

fca_run.tcl fpv_run.tcl fxp_run.tcl
1 set_fml_appmode FXP
2
3 set design async_fifo_dut
4
5 read_file -top $design -format sverilog -sva -vcs {-f ../RTL/filelist}
6
7 create_clock rclk -period 70
8 create_reset rrst_n -sense low
9
10 create_clock wclk -period 100
11 create_reset wrst_n -sense low
12
13 sim_run -stable
14 sim_save_reset
15
16 fxp generate

```

TCL File (Possibility 3: wclk is faster than rclk):

```

fca_run.tcl + fpv_run.tcl fxp_run.tcl
1 set_fml_appmode FPV
2 set design async_fifo_dut
3
4 read_file -top $design -format sverilog -sva -aep all -vcs {-f ../RTL/filelist}
5
6 # Creating clock and reset signals
7 create_clock wclk -period 100 -initial 0
8 create_reset wrst_n -sense low
9
10 # Creating clock and reset signals
11 create_clock rclk -period 70 -initial 0
12 create_reset rrst_n -sense low
13
14 # Running a reset simulation
15 sim_run -stable
16 sim_save_reset
17
18 check fv
19 report_fv

```

FXP Result for all possible cases of wclk and rclk:

The screenshot shows the VCF tool interface. The 'Task List' pane on the left shows a task named 'FXP' with a progress bar. The main pane displays a table of constraints and a summary of the task results.

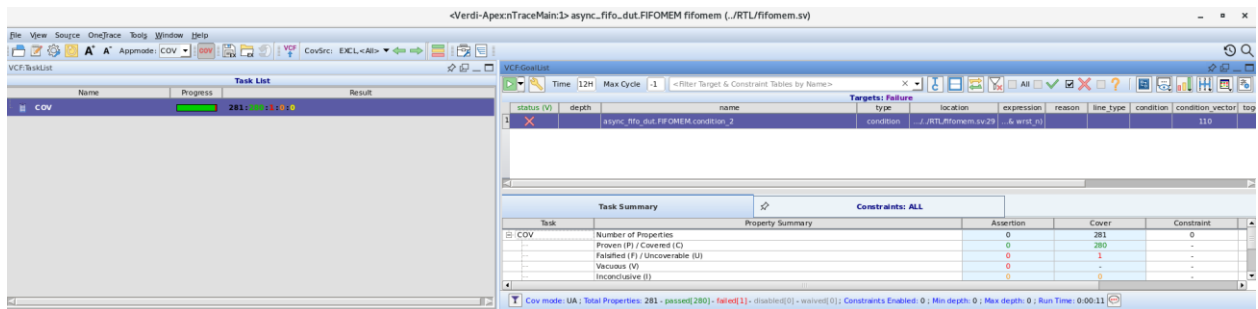
status	depth	name	vacuity	location	rootcause_type	rootcause_signal	engine	elapsed_time
✓	1	._fvp_1_out_state_0		._dut.sv:2	net_computed		klcp_1	00:00:04
✓	2	._fvp_1_out_memory_1		._dut.sv:4	net_computed		t1	00:00:01
✓	3	._fvp_1_out_full_3		._dut.sv:3	net_computed		t1	00:00:01

Task	Property Summary	Assertion	Cover	Constraint	Vacuity	Witness
FXP	Number of Properties	3	0	5	0	0
	Proven (P) / Covered (C)	3	0	-	0	0
	Falsified (F) / Uncoverable (U)	0	0	-	0	0
	Vacuous (V)	0	-	-	-	-
	Inconclusive (I)	0	-	-	0	0
	Checking (CH)	0	0	-	0	0

Total Properties: 3 - passed(3) - failed(0) - disabled(0) ; Constraints Enabled: 5 ; Run Time: 0:00:11

3) FCA Application:

It is efficient to check the coverage while we work on FPV based checking, so we can trace how many of the DUT features have been covered by written assertions.



We found one unreachable combination that was never exercised by DUT as the condition was not possible in any case/condition for DUT, so we removed it to eliminate unreachable part.

DUT Error part:

```

21 */
22 always_ff@(posedge wclk or negedge wrst_n)
23 begin
24     if(!wrst_n)
25     begin
26         foreach(fifo[i])
27             fifo[i] <= 0;
28     end
29     else if (wclken && !wfull && wrst_n)
30     begin
31         fifo[waddr] <= wdata;
32     end
33 end
34

```

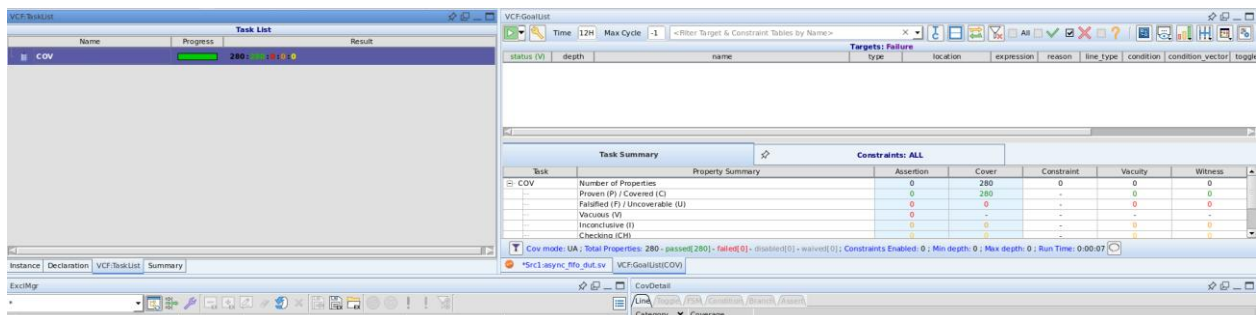
Updated DUT to remove errors:

```

1 */
2 always_ff@(posedge wclk or negedge wrst_n)
3 begin
4     if(!wrst_n)
5     begin
6         foreach(fifo[i])
7             fifo[i] <= 0;
8     end
9     else if (wclken && !wfull)
10    begin
11        fifo[waddr] <= wdata;
12    end
13 end
14

```

Final Coverage (FCA) result:



Conclusion

The formal verification of the asynchronous FIFO design demonstrated the strength, rigor, and completeness of assertion-based methods for validating clock-domain crossing (CDC) structures. Because the write and read domains operate on independent clocks, conventional simulation cannot fully capture all timing configurations or subtle corner cases. By applying System Verilog Assertions together with Synopsys VC Formal, the design was exhaustively explored across every valid sequence of reads, writes, resets, and pointer transitions.

This project deepened our understanding of assertion-based verification and CDC-centric design. Using SVA, FPV, AEP, FXP, and formal coverage, we validated the FIFO's functional integrity and ensured it behaves correctly across clock domains. The verified properties guarantee that the FIFO meets all essential design requirements, including:

- No overflow or underflow
- Correct first-in, first-out data ordering
- Stable and monotonic synchronizer behaviour across domains
- Accurate full and empty flag generation based on pointer comparisons
- Safe reset, initialization, and power-up behaviour

All safety properties were proven, and cover properties demonstrated key reachability scenarios, such as pointer wrap-around, FIFO transitions from empty to full, and concurrent read/write operations. Bug-injection exercises further revealed that small deviations in pointer logic or flag computation can lead to subtle but critical failures—showing the effectiveness of formal verification in identifying issues that simulation might miss.

This project provided hands-on experience with industry-grade formal tools and strengthened practical skills in formal planning, constraint modelling, debugging counterexamples, and documenting verification methodology. Overall, the work confirms that formal verification is an essential approach for validating designs with asynchronous interfaces, offering exhaustive confidence in correctness while reducing the need for extensive simulation testbenches.

Credits / Citing / References

1. **Sunburst Design – Asynchronous FIFO Design** *Clifford Cummings, Peter Alfke*
SNUG San Jose, 2002
2. RTL taken from Previous coursework of Pre silicon validation.
3. ChatGPT